

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Azfarafi Gustiar Jati - 5024241037

2026

1 Langkah-Langkah Percobaan

1.1 PERCOBAAN 1: Pengenalan STM32 Blackpill (Blink)

1. Ruang kerja proyek inisial dibuka melalui perangkat lunak VSCode, lalu parameter berkas konfigurasi `platformio.ini` disetel agar menargetkan tipe *board* STM32 Blackpill (`blackpill_f401cc`) dengan basis *framework* Arduino serta protokol komunikasi *stlink*:

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

   Catatan: Setiap kali pembaruan konfigurasi dilakukan pada `platformio.ini`, PlatformIO secara otomatis menjalankan proses penyegaran indeks sistem. Pastikan proses tersebut berakhir sepenuhnya sebelum mengeksekusi perintah *Build* atau *Upload*.

2. Alat pemrogram eksternal ST-Link V2 dihubungkan menuju pin header antarmuka *Serial Wire Debug* (SWD) yang terletak pada *board* mikrokontroler menggunakan empat kabel jumper jenis *female-female* dengan ketentuan jalur:

   - SWDIO pada ST-Link ditautkan ke pin SWDIO board (saluran data serta debug).
   - SWCLK pada ST-Link ditautkan ke pin SWCLK board (saluran pewaktuan/clock).
   - GND pada ST-Link ditautkan ke pin GND board (potensial referensi bersama).
   - Tegangan 3.3V pada ST-Link ditautkan ke pin 3V3 board (penyedia catu daya utama).

3. Verifikasi fisik jalur perkabelan dilakukan untuk memastikan suplai daya berada pada level 3.3 V sebelum perangkat ST-Link dihubungkan ke komputer via soket USB.

4. Naskah instruksi kode program penayangan kedipan LED disusun di dalam direktori `src/main.cpp` melalui baris berikut:

```
#include <Arduino.h>

#define LED_PIN PC13

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW -> LOW berarti menyala
  delay(500);
  digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
  delay(500);
}
```

5. Tahapan kompilasi dan pengunggahan kode dijalankan, diikuti pengamatan visual pada indikator LED fisik pin PC13 yang berkedip dengan interval setengah detik secara berkelanjutan.

1.2 PERCOBAAN 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Pengaturan berkas `platformio.ini` dimodifikasi guna mengaktifkan lingkungan pengembangan *ESP-IDF* pada perangkat keras ESP32 DevKit:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

2. Modul ESP32 dikoneksikan ke komputer menggunakan kabel data USB, dan ketersediaan port komunikasi serial diverifikasi pada sistem operasi.

3. Berkas kode lama `src/main.cpp` disingkirkan, lalu digantikan dengan pembuatan berkas baru bertaraf bahasa C bernama `src/main.c` yang memuat sintaksis pemrograman tingkat rendah berbasis *ESP-IDF*:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void) {
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1) {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

4. Eksekusi kompilasi dan pengiriman biner dilakukan ke chip ESP32, lalu karakteristik arsitektur dibandingkan terhadap struktur *framework* Arduino.

1.3 PERCOBAAN 3: Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Arduino)

1. Lingkungan `platformio.ini` dikembalikan ke konfigurasi baku berbasis *framework* Arduino pada platform ESP32:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

2. Sepasang tombol tekan (*pushbutton*) dirangkai menuju pin GPIO 32 serta GPIO 33 dengan menyertakan resistor eksternal sebesar 10 kΩ (tombol pertama memakai skema pull-up eksternal ke 3.3 V, sedangkan tombol kedua memakai skema pull-down eksternal ke GND).

3. Kode sumber evaluasi pertama ditanamkan ke dalam berkas `src/main.cpp`, dilanjutkan proses *Build* dan *Upload* guna merekam respons status logika tombol melalui layar *Serial Monitor*.

4. Seluruh komponen resistor eksternal dicabut dari papan rangkaian tanpa mengubah posisi kontak tombol, lalu kode program digantikan dengan implementasi internal resistor melalui Kode B berikut:

```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32
#define BUTTON_PULLDOWN_PIN 33

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN);
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[INTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

5. Pengujian ulang dijalankan untuk mengevaluasi keselarasan luaran antara penerapan resistor eksternal dan pemanfaatan fitur internal mikrokontroler.

1.4 PERCOBAAN 4: Debouncing pada Input Tombol GPIO (ESP32 + Arduino)

1. Sebuah tombol mekanik ditautkan pada pin GPIO 25 dengan bantuan konfigurasi mode penarikan *pull-up* internal milik ESP32.

2. Berkas utama `src/main.cpp` diisi ulang menggunakan skema program yang memvalidasi kestabilan pulsa sinyal input guna meredam efek pantulan mekanis (*bouncing*):

```
#include <Arduino.h>

#define BUTTON_PIN 25
#define DEBOUNCE_DELAY 50

int lastFlickerableState = HIGH;
int stableState = HIGH;
unsigned long lastDebounceTime = 0;
unsigned long counterWithDebounce = 0;

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
```

```
  int reading = digitalRead(BUTTON_PIN);

  if (reading != lastFlickerableState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (reading != stableState) {
      stableState = reading;
      if (stableState == LOW) {
        counterWithDebounce++;
      }
    }
  }
  lastFlickerableState = reading;

  delay(10);
}
```

3. Aktivasi fisik tombol dilakukan secara berulang untuk memantau perubahan akumulasi hitungan pada *Serial Monitor*. Nilai tetapan penundaan (`DEBOUNCE_DELAY`) divariasikan (meliputi 10 ms, 50 ms, hingga 200 ms) untuk mengukur tingkat efektivitas filter terhadap jeda respons sakelar.

1.5 PERCOBAAN 5: Akses GPIO Analog: Pembacaan ADC dengan LDR

1. Rangkaian pembagi tegangan eksternal dibangun dengan mengombinasikan sensor LDR dan resistor 10 kΩ, di mana titik temu kedua komponen dihubungkan langsung ke pin masukan analog GPIO 34 pada ESP32.

2. Logika perangkat lunak untuk mengambil data konversi analog ke digital serta menerjemahkannya ke dalam besaran tegangan dituliskan ke dalam berkas `src/main.cpp`:

```
#include <Arduino.h>

#define LDR_PIN 34

void setup() {
  Serial.begin(115200);
}

void loop() {
  int raw = analogRead(LDR_PIN);
  float voltage = raw * (3.3 / 4095.0);

  Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
  delay(300);
}
```

3. Rangkaian instruksi dikompilasi dan dikirimkan ke modul mikrokontroler, kemudian fluktuasi paparan cahaya pada sensor LDR diamati melalui rentang data digital (0 sampai 4095) yang tercetak pada *Serial Monitor*.

2 Analisis Hasil Percobaan

2.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Program pengenalan *board* STM32 Blackpill berhasil dikompilasi serta diunggah menggunakan protokol ST-Link dengan memanfaatkan kerangka kerja Arduino pada STM32duino. Indikator LED fisik yang terhubung pada pin PC13 berkedip secara periodik dengan interval waktu setengah detik menyala dan setengah detik mati sesuai instruksi penundaan *delay*. Keberhasilan pengujian ini membuktikan bahwa pemetaan pin fisik dapat dikenali secara langsung tanpa perlu melakukan konfigurasi register *GPIO_InitTypeDef* secara manual seperti pada arsitektur tingkat rendah. Faktor penentu keberhasilan pada percobaan ini terletak pada ketepatan instalasi berkas *platformio.ini* serta kestabilan koneksi fisik kabel pada jalur *Serial Wire Debug*.

2.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF

Pemrograman tingkat rendah menggunakan kerangka kerja resmi *ESP-IDF* pada modul ESP32 berhasil dijalankan untuk mengendalikan kedipan LED pada pin GPIO 2. Berdasarkan analisis arsitektural, penggunaan *ESP-IDF* menuntut pemahaman mendalam mengenai manajemen pustaka sistem operasi waktu nyata *FreeRTOS* melalui fungsi utama *app_main* yang berjalan sebagai sebuah *task*. Pengaturan arah pin dan level tegangan dilakukan secara eksplisit menggunakan fungsi driver *gpio_set_direction* serta *gpio_set_level*, sementara penundaan non-blokir memanfaatkan *vTaskDelay*. Percobaan ini menunjukkan bahwa pengelolaan perangkat keras secara native memberikan fleksibilitas tinggi dalam pengembangan sistem tertanam yang kompleks.

2.3 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal

Pengujian sistem pembacaan tombol digital membuktikan bahwa penggunaan resistor penarik *pull-up* dan *pull-down* sangat krusial untuk mencegah kondisi *floating* pada pin masukan mikrokontroler. Berdasarkan data yang tercatat pada *Serial Monitor*, baik menggunakan resistor eksternal 10 kΩ maupun memanfaatkan konfigurasi resistor internal ESP32 (*INPUT_PULLUP* dan *INPUT_PULLDOWN*), hasil pembacaan logika menunjukkan perilaku yang sangat konsisten. Pin *pull-up* bernilai HIGH saat tombol dilepas dan berubah menjadi LOW saat ditekan, sedangkan pin *pull-down* berlaku sebaliknya sesuai kaidah rangkaian pembagi tegangan. Hal ini memvalidasi efisiensi perangkat keras modern yang telah menyediakan resistor internal sehingga menyederhanakan tata letak rangkaian fisik di luar mikrokontroler.

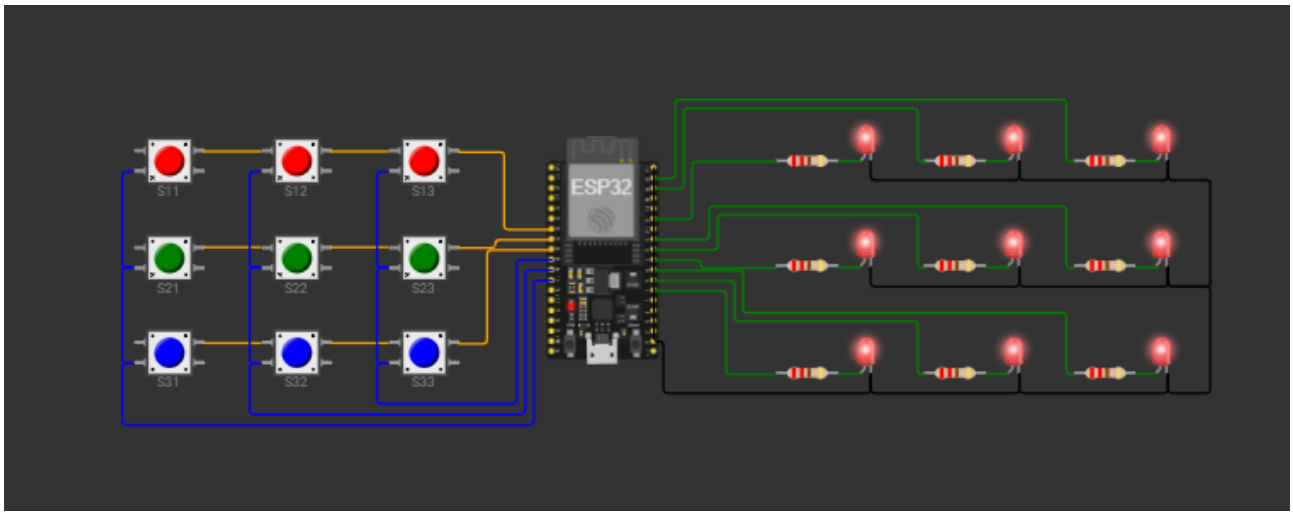
2.4 Percobaan 4: Debouncing pada Input Tombol GPIO

Identifikasi permasalahan getaran mekanik atau *bouncing* pada tombol taktil berhasil dilakukan melalui pemantauan nilai penghitung tanpa dan dengan penyaring perangkat lunak. Tanpa adanya penundaan waktu, lonjakan transisi sinyal listrik yang terjadi saat tombol ditekan menyebabkan pencatatan ganda yang tidak akurat pada variabel *counter*. Implementasi algoritma debouncing berbasis fungsi waktu *millis* dan konstanta *DEBOUNCE_DELAY* terbukti efektif meredam sinyal palsu tersebut sehingga transisi stabil dapat divalidasi. Analisis lebih lanjut menunjukkan bahwa pemilihan nilai penundaan harus mempertimbangkan keseimbangan antara kebersihan sinyal dan kecepatan respons tombol terhadap interaksi pengguna.

2.5 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR

Percobaan pembacaan sinyal analog melalui sensor LDR dan pin ADC ESP32 belum dapat diselesaikan dikarenakan keterbatasan waktu pelaksanaan sesi laboratorium yang tersedia. Akibat kendala tersebut, pengujian konversi tingkat intensitas cahaya menjadi nilai digital 12-bit beserta perhitungannya dalam satuan volt belum sempat diverifikasi secara tuntas pada perangkat keras. Meskipun demikian, secara teoretis pin GPIO 34 pada ESP32 mendukung fungsi ADC kanal 1 yang handal untuk membaca variasi tegangan kontinyu. Kendala waktu ini menjadi evaluasi penting untuk meningkatkan manajemen durasi pada pelaksanaan praktikum pengantar mikrokontroler berikutnya.

3 Hasil Tugas Modul



Gambar 1: Simulasi Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3

```
Tombol (1,3) DITEKAN   | perubahan sinyal mentah:  3 | LED (1,3) NYALA
Tombol (1,3) DILEPAS   | perubahan sinyal mentah:  1 | LED (1,3) NYALA
Tombol (1,3) DITEKAN   | perubahan sinyal mentah:  1 | LED (1,3) MATI
Tombol (1,3) DILEPAS   | perubahan sinyal mentah:  3 | LED (1,3) MATI
Tombol (1,3) DITEKAN   | perubahan sinyal mentah:  3 | LED (1,3) NYALA
Tombol (1,3) DILEPAS   | perubahan sinyal mentah:  1 | LED (1,3) NYALA
Tombol (1,1) DITEKAN   | perubahan sinyal mentah:  1 | LED (1,1) MATI
Tombol (1,1) DILEPAS   | perubahan sinyal mentah:  3 | LED (1,1) MATI
Tombol (1,2) DITEKAN   | perubahan sinyal mentah:  1 | LED (1,2) MATI
```

Gambar 2: Bukti Debouncing

Simulasi ini menggunakan mikrokontroler ESP32 untuk mengontrol Matriks LED 3x3 menggunakan input dari Matriks Pushbutton 3x3. Konfigurasi perangkat keras dirancang agar tombol pada posisi baris dan kolom tertentu dapat menyalakan LED pada koordinat yang sama secara akurat. Pemindaian tombol dilakukan secara matriks melalui tiga pin baris dan tiga pin kolom untuk efisiensi penggunaan pin GPIO.

Program ini mengimplementasikan algoritma penyaringan sinyal (*debouncing*) berbasis fungsi waktu *millis()* untuk masing-masing tombol secara individual. Ketika tombol ditekan, sinyal mentah tidak langsung mengubah status LED, melainkan divalidasi hingga stabil selama interval waktu tertentu guna mengeliminasi gangguan akibat pantekan kontak mekanik. Selain itu, Serial Monitor secara aktif memantau dan mencatat jumlah perubahan sinyal mentah (*bouncing*) yang terjadi sebelum status stabil diterima oleh sistem.

Berikut adalah kode sumber lengkap untuk program matriks tombol dan LED tersebut:

```
#include <Arduino.h>

const uint8_t ROW_PINS[3] = {32, 33, 25};
const uint8_t COL_PINS[3] = {26, 27, 14};

const uint8_t LED_PINS[3][3] = {
  {23, 22, 21},
  {19, 18,  5},
  {17, 16,  4}
};

const unsigned long DEBOUNCE_MS = 50;
#define MODE_TOGGLE 1

struct Tombol {
  bool bacaanTerakhir;
  bool statusStabil;
  unsigned long waktuBerubah;
  uint16_t jumlahPerubahan;
};

Tombol tombol[3][3];
bool statusLED[3][3];

void pindaiBaris(uint8_t r, bool hasil[3]) {
  digitalWrite(ROW_PINS[r], LOW);
  pinMode(ROW_PINS[r], OUTPUT);
  delayMicroseconds(10);

  for (uint8_t c = 0; c < 3; c++) {
    hasil[c] = (digitalRead(COL_PINS[c]) == LOW);
  }

  pinMode(ROW_PINS[r], INPUT);
}

void prosesDebounce(uint8_t r, uint8_t c, bool bacaan) {
  Tombol &t = tombol[r][c];
  unsigned long sekarang = millis();

  if (bacaan != t.bacaanTerakhir) {
    t.waktuBerubah = sekarang;
    t.bacaanTerakhir = bacaan;
    t.jumlahPerubahan++;
  }

```

```
  if ((sekarang - t.waktuBerubah) >= DEBOUNCE_MS && bacaan != t.statusStabil) {
    t.statusStabil = bacaan;

#if MODE_TOGGLE
    if (t.statusStabil) {
      statusLED[r][c] = !statusLED[r][c];
    }
#else
    statusLED[r][c] = t.statusStabil;
#endif

    digitalWrite(LED_PINS[r][c], statusLED[r][c] ? HIGH : LOW);

    Serial.printf("Tombol (%u,%u) %-8s | perubahan sinyal mentah: %2u | LED (%u,%u) %
    s\n",
                  r + 1, c + 1,
                  t.statusStabil ? "DITEKAN" : "DILEPAS",
                  t.jumlahPerubahan,
                  r + 1, c + 1,
                  statusLED[r][c] ? "NYALA" : "MATI");

    t.jumlahPerubahan = 0;
  }
}

void setup() {
  Serial.begin(115200);

  for (uint8_t i = 0; i < 3; i++) {
    pinMode(ROW_PINS[i], INPUT);
    pinMode(COL_PINS[i], INPUT_PULLUP);
  }

  for (uint8_t r = 0; r < 3; r++) {
    for (uint8_t c = 0; c < 3; c++) {
      digitalWrite(LED_PINS[r][c], LOW);
      pinMode(LED_PINS[r][c], OUTPUT);
      statusLED[r][c] = false;
      tombol[r][c] = {false, false, 0, 0};
    }
  }

  Serial.println();
  Serial.println("=== Matrix Pushbutton 3x3 -> Matrix LED 3x3 (debounced) ===");
  Serial.printf("Debounce: %lu ms | Mode: %s\n", DEBOUNCE_MS,
                MODE_TOGGLE ? "TOGGLE" : "MOMENTARY");
}

void loop() {
  bool hasil[3];

  for (uint8_t r = 0; r < 3; r++) {
    pindaiBaris(r, hasil);
    for (uint8_t c = 0; c < 3; c++) {
```

```
100         prosesDebounce(r, c, hasil[c]);
101       }
102     }
103 }
```

4 Kesimpulan

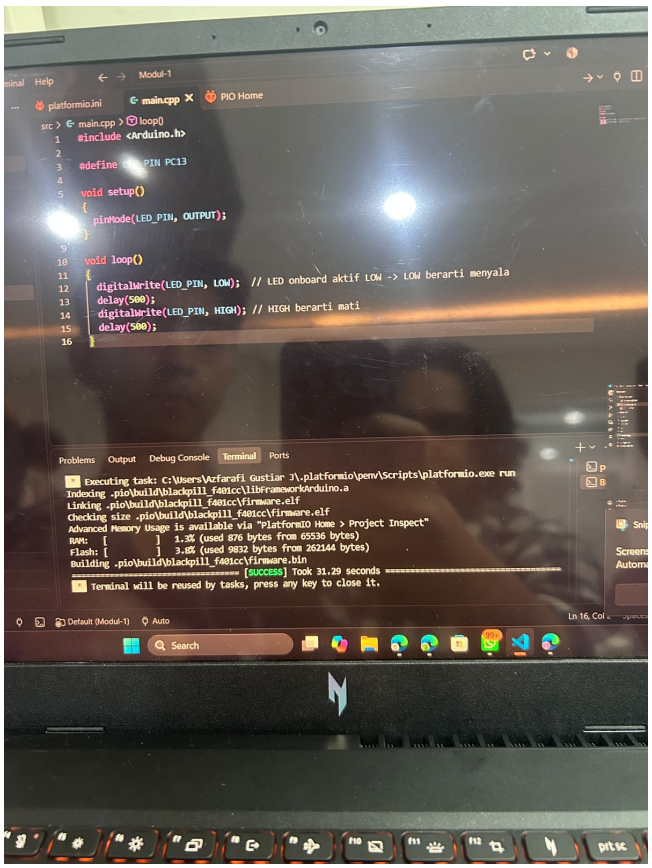
Rangkaian kegiatan praktikum yang telah dilaksanakan berhasil memvalidasi konsep dasar pemrograman antarmuka GPIO digital pada arsitektur mikrokontroler STM32 dan ESP32 menggunakan kerangka kerja Arduino serta ESP-IDF, mulai dari pengendalian keluaran LED hingga pemindaian matriks tombol 3x3. Seluruh hasil pengujian pada percobaan pertama hingga keempat serta percobaan tambahan menunjukkan kesesuaian dengan teori, khususnya pada efektivitas resistor *pull-up*/*pull-down* internal maupun eksternal dalam mencegah kondisi *floating* serta penerapan algoritma *debouncing* berbasis waktu *millis()* untuk meredam getaran kontak mekanik. Meskipun pengujian pembacaan masukan analog ADC menggunakan sensor LDR pada percobaan kelima belum dapat diselesaikan secara fisik akibat keterbatasan durasi pelaksanaan laboratorium, keseluruhan kegiatan ini memberikan pemahaman komprehensif mengenai konfigurasi perangkat keras tertanam, manajemen waktu eksekusi non-blokir, serta pentingnya efisiensi pengelolaan waktu selama pengujian sistem.

5 Lampiran

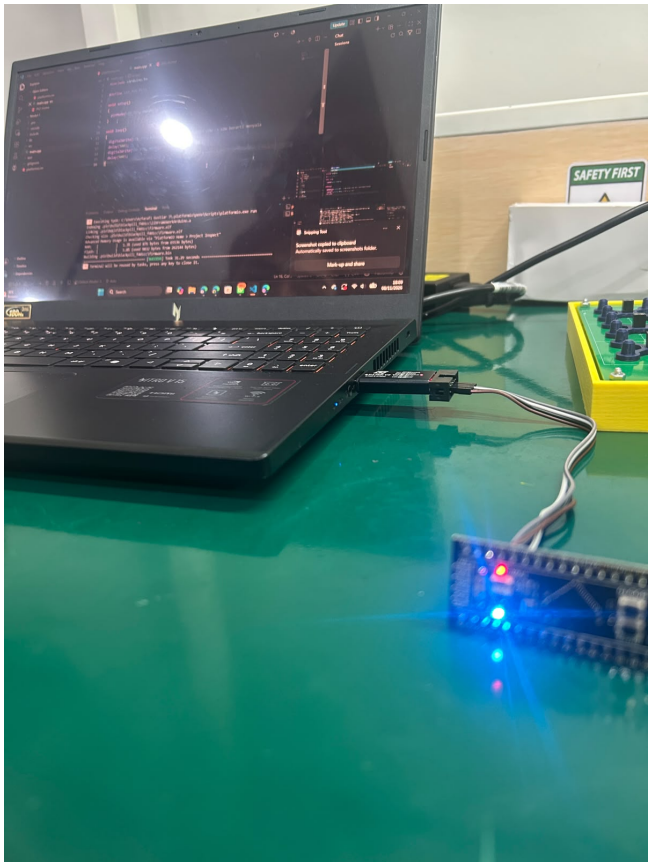
5.1 Dokumentasi saat praktikum



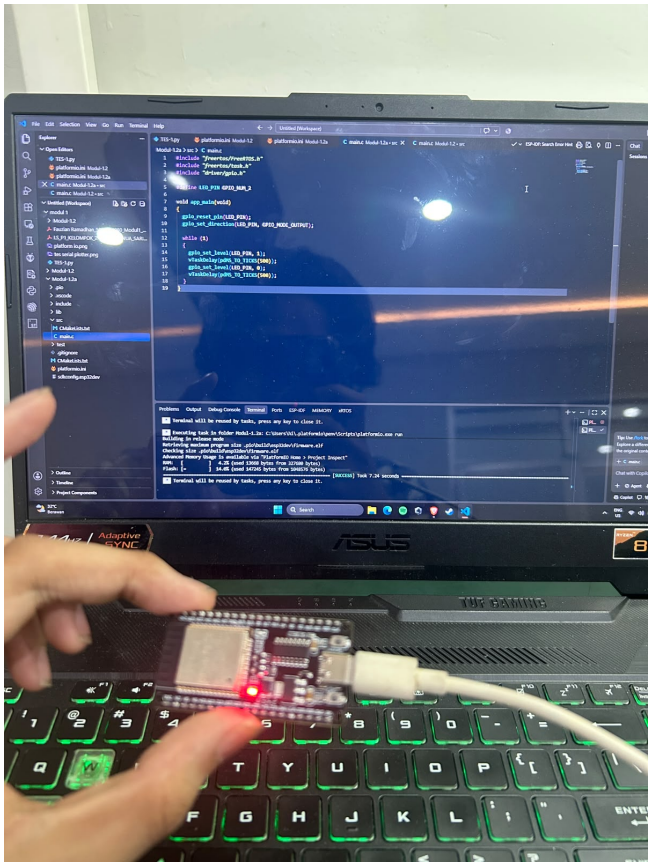
Gambar 3: Alat dan Bahan



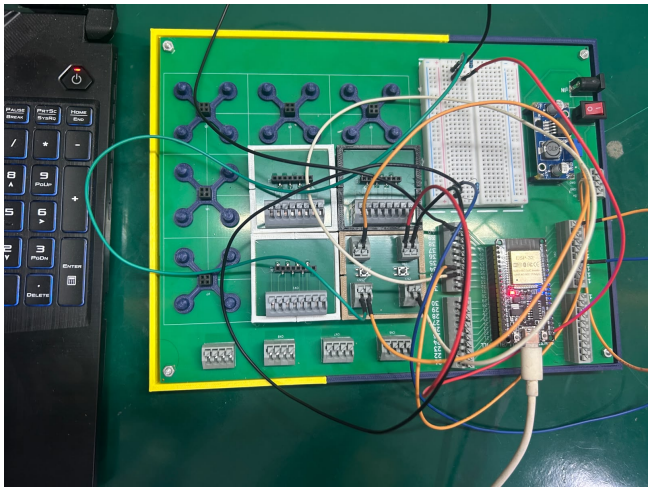
Gambar 4: Percobaan 1



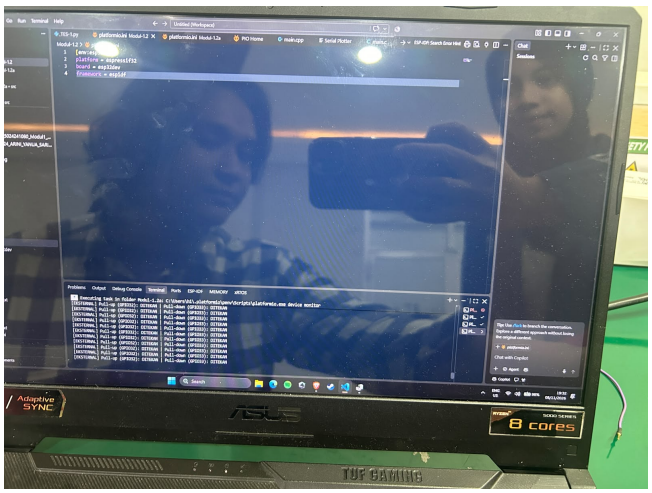
Gambar 5: Percobaan 1



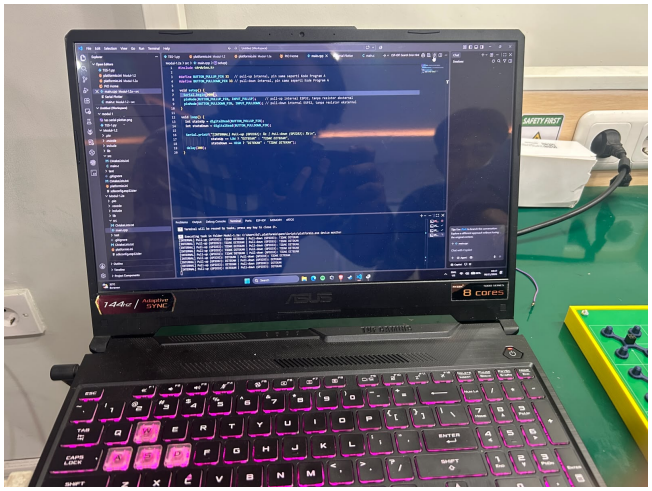
Gambar 6: Percobaan 2



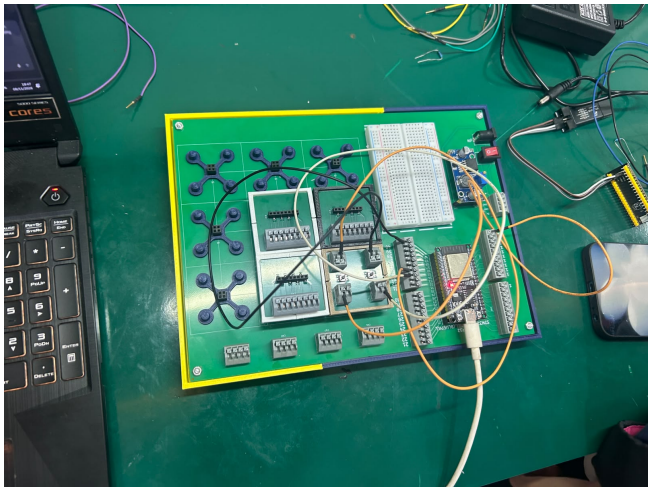
Gambar 7: Percobaan 3



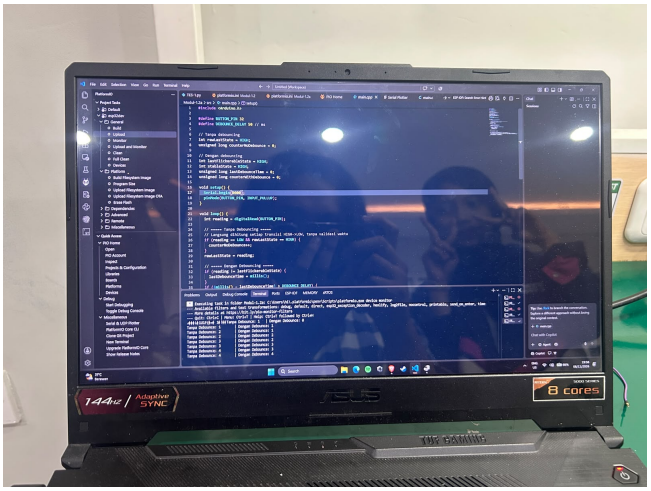
Gambar 8: Percobaan 3a



Gambar 9: Percobaan 3b



Gambar 10: Percobaan 4



Gambar 11: Percobaan 4



Gambar 12: Bukti Mengikuti Praktikum